SpaceX Mission Analytics: Launches, Landings and Reusability

A MongoDB & Python Analysis of the SpaceX REST API

Azeglio Martinelli

2026-03-21

Table of contents

1 Summary **1**

2 Research Questions **2**

3 Requirements & Configuration **2**

4 ELT Process **3**
4.1 DB Setup . . . 3
4.2 Raw Data Loading . . . 4
4.3 Data Inspection . . . 6
4.4 Data Cleaning & Transformation . . . 16

5 Class Diagramm **21**

6 Analysis **22**
6.1 Launch success rate by rocket . . . 22
6.2 Most active launchpads . . . 24
6.3 Landing success rate by landing pad type . . . 27
6.4 Launch frequency over time . . . 30
6.5 Core reuse analysis . . . 33
6.6 Fail rate . . . 35

1 Summary

This project answers research questions related to SpaceX API data using MongoDB as data storage. In order to do that several API's we fetched and the raw data stored in loading collections. Then the raw data was cleaned, transformed and stored in staging collections. For analysis the data could have been stored in a final, unified collection combining the staged data collections. However, to increase the complexity of the aggregations and pipelines, the data was combined using lookups to answer the

research questions. All pipelines use the pymongo package as much as possible, after each analysis pipeline a visual, created with mostly matplotlib, was plotted.

2 Research Questions

The following research questions are answered in this project:

1. How has SpaceX's launch success rate evolved over time, and which rocket types are most reliable?
2. Which launch sites are most active, and how does location affect launch success?
3. How successful is SpaceX at recovering rocket boosters, and which landing infrastructure performs best?

3 Requirements & Configuration

First, to run the code, the necessary Python packages from the requirements.txt are installed and then imported.

```
import sys
!{sys.executable} -m pip install -r requirements.txt

import pymongo
from pymongo.errors import ConnectionFailure
import pprint as pp
import pandas as pd
import requests
import json
import time
import string
import os
import matplotlib.pyplot as plt
# to render labels correctly: use a regular ASCII hyphen
plt.rcParams['text.parse_math'] = False
from dotenv import load_dotenv
```

Next, important secrets or keys are imported from the .env file. The configuration of the database and APIs as well as a folder to store images is set up.

```
# API and Database details
load_dotenv()

# MongoDB
CNX_STR = os.getenv("MONGO_URI")
DB_NAME = "spacexdb"

# SpaceX API endpoints
```

```python
LAUNCHES_URL = "https://api.spacexdata.com/v5/launches"
ROCKETS_URL = "https://api.spacexdata.com/v4/rockets"
LAUNCHPADS_URL = "https://api.spacexdata.com/v4/launchpads"
LANDINGPADS_URL = "https://api.spacexdata.com/v4/landpads"

# for images
os.makedirs("Personal_project_files/images", exist_ok=True)
```

4 ELT Process

4.1 DB Setup

Before any data is loaded, the connection to MongoDB and the database variable are defined. Then the variables to store the raw data from the API endpoints and the staging collections are defined.

```python
# Connection to MongoDB
client = pymongo.MongoClient(CNX_STR)
db = client[DB_NAME]

launches_raw = db["launches_raw"]
rockets_raw = db["rockets_raw"]
launchpads_raw = db["launchpads_raw"]
landingpads_raw = db["landingspads_raw"]

launches_stage = db["launches_stage"]
rockets_stage = db["rockets_stage"]
launchpads_stage = db["launchpads_stage"]
landingpads_stage = db["landingpads_stage"]
```

The cell below, tests the connection to the defined client from the cell above. It is a quick way to test if the database and its collections are available.

```python
# Test connection
try:
    # The ping command is cheap and does not require auth.
    client.admin.command('ping')
    print('Ping successful!')
    print("Available collections:")
    for name in sorted(db.list_collection_names()):
        print(f"  - {name}")
except ConnectionFailure:
    print("Server not available!")
```

```
Ping successful!
Available collections:
```

```
  - landingpads_stage
  - landingspads_raw
  - launches_raw
  - launches_stage
  - launchpads_raw
  - launchpads_stage
  - rockets_raw
  - rockets_stage
```

To avoid storing duplicate data entries when calling the API endpoints, the cell below deletes any data stored in the collections. This must not be run everytime but is useful when converting the file to pdf when run with Quarto. Note that this would not be done in a professional database environment.

```python
# OPTIONAL: delete raw data
launches_raw.drop()
rockets_raw.drop()
launchpads_raw.drop()
landingpads_raw.drop()

print("launches_raw:", launches_raw.count_documents({}))
print("rockets_raw:", rockets_raw.count_documents({}))
print("launchpads_raw:", launchpads_raw.count_documents({}))
print("landingpads_raw:", landingpads_raw.count_documents({}))
```

```
launches_raw: 0
rockets_raw: 0
launchpads_raw: 0
landingpads_raw: 0
```

4.2 Raw Data Loading

Following the completed configuration steps, the raw dat is fetched from the API endpoints and stored in the collections defined as "raw". For each collection, the outputs show how many documents were received when calling the API and how many documents are stored in the respective collection. Ideally, each output pair should show the same number. Else this security mechanism indicates that duplicate entries are stored or further investigation is needed.

```python
# Load raw launches data into launches_raw
response = requests.get(LAUNCHES_URL)
response.raise_for_status()

launches_data = response.json()

print(f"Launches downloaded: {len(launches_data)}")

if launches_data:
    launches_raw.insert_many(launches_data)
```

```python

print("Documents in launches_raw:", launches_raw.count_documents({}))

# Load raw rockets data into rockets_raw
response = requests.get(ROCKETS_URL)
response.raise_for_status()

rockets_data = response.json()

print(f"Rockets downloaded: {len(rockets_data)}")

if rockets_data:
    rockets_raw.insert_many(rockets_data)

print("Documents in rockets_raw:", rockets_raw.count_documents({}))

# Load raw launchpads data into launchpads_raw
response = requests.get(LAUNCHPADS_URL)
response.raise_for_status()

launchpads_data = response.json()

print(f"Launch pads downloaded: {len(launchpads_data)}")

if launchpads_data:
    launchpads_raw.insert_many(launchpads_data)

print("Documents in launchpads_raw:", launchpads_raw.count_documents({}))

# Load raw landingpads data into landingpads_raw
response = requests.get(LANDINGPADS_URL)
response.raise_for_status()

landingpads_data = response.json()

print(f"Launch pads downloaded: {len(landingpads_data)}")

if landingpads_data:
    landingpads_raw.insert_many(landingpads_data)

print("Documents in landingpads_raw:", landingpads_raw.count_documents({}))
```

```
Launches downloaded: 205
Documents in launches_raw: 205
Rockets downloaded: 4
Documents in rockets_raw: 4
Launch pads downloaded: 6
Documents in launchpads_raw: 6
```

```
Launch pads downloaded: 7
Documents in landingpads_raw: 7
```

4.3 Data Inspection

Before any transformation is done, some inspection pipelines on the raw data are carried out. These inspections are necessary to define the transformation steps for the staging pipelines.

The cell below, prints out one document and some fields for each raw collection. This serves as a sanity check to see if any anomalies exist but also to get a direction where to dive deeper with the inspection.

```python
# Quick sanity check
for label, doc in [
    (
        "launches_raw",
        launches_raw.find_one(
            {}, {"_id": 0, "name": 1, "date_utc": 1, "rocket": 1, "launchpad": 1}
        ),
    ),
    (
        "rockets_raw",
        rockets_raw.find_one(
            {}, {"_id": 0, "name": 1, "cost_per_launch": 1, "success_rate_pct": 1}
        ),
    ),
    (
        "launchpads_raw",
        launchpads_raw.find_one(
            {}, {"_id": 0, "name": 1, "full_name": 1, "locality": 1, "region": 1}
        ),
    ),
    (
        "landingpads_raw",
        landingpads_raw.find_one(
            {}, {"_id": 0, "name": 1, "full_name": 1, "locality": 1, "region": 1}
        ),
    ),
]:
    print(f"{label}:")
    for k, v in doc.items():
        print(f"  {k}: {v}")
    print()
```

```
launches_raw:
  rocket: 5e9d0d95eda69955f709d1eb
  launchpad: 5e9e4502f5090995de566f86
  name: FalconSat
  date_utc: 2006-03-24T22:30:00.000Z
```

```
rockets_raw:
  name: Falcon 1
  cost_per_launch: 6700000
  success_rate_pct: 40

launchpads_raw:
  name: VAFB SLC 3W
  full_name: Vandenberg Space Force Base Space Launch Complex 3W
  locality: Vandenberg Space Force Base
  region: California

landingpads_raw:
  name: LZ-1
  full_name: Landing Zone 1
  locality: Cape Canaveral
  region: Florida
```

The sanity check confirms that all four raw collections contain data in the expected structure. Notably, the `launches_raw` collection stores `rocket` and `launchpad` as string IDs (e.g. `5e9d0d95eda69955f709d1eb`) rather than embedded documents. This means that during analysis, `$lookup` stages will be needed to join launch data with rocket and launchpad details.

```python
# check for null/missing fields
important_fields = [
    "id",
    "name",
    "date_utc",
    "success",
    "rocket",
    "launchpad",
    "cores",
    "payloads",
]

for field in important_fields:
    missing_or_null = launches_raw.count_documents(
        {"$or": [{field: {"$exists": False}}, {field: None}]}
    )
    print(f"{field}: {missing_or_null}")
```

```
id: 0
name: 0
date_utc: 0
success: 19
rocket: 0
launchpad: 0
cores: 0
```

```
payloads: 0
```

All critical fields like `id`, `name`, `date_utc`, `rocket`, `launchpad`, and `cores` are present in every document. However, the `success` field is null in 19 documents. Closer inspection of these 19 documents can be done with the cell below but for sake of brevity it is not evaluated.

```
# inspect launches with null success values
pp.pprint(list(launches_raw.aggregate([
    {
        "$match": {"success": None}
    },
    {
        "$project": {
            "_id": 0,
            "name": 1,
            "date_utc": 1,
            "success": 1,
            "upcoming": 1,
            "flight_number": 1
        }
    },
    {
        "$sort": {"date_utc": -1}
    }
])))
```

Upon inspection, these 19 documents exhibit strong inconsistencies. Some still have `upcoming: true` despite their scheduled dates being in the past, while others have `upcoming: false` but lack outcome data. In both cases, the success field was never populated. All 19 launches with `success: null` fall within a narrow time window from July to December 2022, and 18 out of 19 are still marked as `upcoming: true`. This aligns with the fact that the SpaceX-API GitHub project has not been updated in over four years, which likely means the data contains errors. Therefore, the raw data cannot be fully trusted. Rather than attempting to manually correct these entries, the decision was made to retain all documents in the database and instead filter out unreliable records at the aggregation level using `{"success": {"$ne": None}}`. This ensures that no assumptions are introduced during the cleaning phase while still producing accurate analysis results.

```
# check date quality
pp.pprint(list(launches_raw.aggregate([
    {
        "$project": {
            "name": 1,
            "date_utc": 1,
            "date_type": {"$type": "$date_utc"}
        }
    },
    {"$limit": 10}
])))
```

```
[{'_id': ObjectId('69beeada1eb88c8f9b0d78f8'),
  'date_type': 'string',
  'date_utc': '2006-03-24T22:30:00.000Z',
  'name': 'FalconSat'},
 {'_id': ObjectId('69beeada1eb88c8f9b0d78f9'),
  'date_type': 'string',
  'date_utc': '2007-03-21T01:10:00.000Z',
  'name': 'DemoSat'},
 {'_id': ObjectId('69beeada1eb88c8f9b0d78fa'),
  'date_type': 'string',
  'date_utc': '2008-08-03T03:34:00.000Z',
  'name': 'Trailblazer'},
 {'_id': ObjectId('69beeada1eb88c8f9b0d78fb'),
  'date_type': 'string',
  'date_utc': '2008-09-28T23:15:00.000Z',
  'name': 'RatSat'},
 {'_id': ObjectId('69beeada1eb88c8f9b0d78fc'),
  'date_type': 'string',
  'date_utc': '2009-07-13T03:35:00.000Z',
  'name': 'RazakSat'},
 {'_id': ObjectId('69beeada1eb88c8f9b0d78fd'),
  'date_type': 'string',
  'date_utc': '2010-06-04T18:45:00.000Z',
  'name': 'Falcon 9 Test Flight'},
 {'_id': ObjectId('69beeada1eb88c8f9b0d78fe'),
  'date_type': 'string',
  'date_utc': '2010-12-08T15:43:00.000Z',
  'name': 'COTS 1'},
 {'_id': ObjectId('69beeada1eb88c8f9b0d78ff'),
  'date_type': 'string',
  'date_utc': '2012-05-22T07:44:00.000Z',
  'name': 'COTS 2'},
 {'_id': ObjectId('69beeada1eb88c8f9b0d7900'),
  'date_type': 'string',
  'date_utc': '2012-10-08T00:35:00.000Z',
  'name': 'CRS-1'},
 {'_id': ObjectId('69beeada1eb88c8f9b0d7901'),
  'date_type': 'string',
  'date_utc': '2013-03-01T19:10:00.000Z',
  'name': 'CRS-2'}]
```

The `date_utc` field is stored as a string (type `"string"`) rather than as a BSON Date object. This prevents the use of MongoDB's date operators such as `$year` or `$month` directly on the raw data. During transformation, `$dateFromString` will be used to convert these strings into proper Date objects, enabling time-based aggregations.

```
# inspect array sizes of cores and payloads
pp.pprint(list(launches_raw.aggregate([
```

```
    {
        "$project": {
            "name": 1,
            "core_count": {"$size": {"$ifNull": ["$cores", []]}},
            "payload_count": {"$size": {"$ifNull": ["$payloads", []]}}
        }
    },
    {"$limit": 10}
]))
```

```
[{'_id': ObjectId('69beeada1eb88c8f9b0d78f8'),
  'core_count': 1,
  'name': 'FalconSat',
  'payload_count': 1},
 {'_id': ObjectId('69beeada1eb88c8f9b0d78f9'),
  'core_count': 1,
  'name': 'DemoSat',
  'payload_count': 1},
 {'_id': ObjectId('69beeada1eb88c8f9b0d78fa'),
  'core_count': 1,
  'name': 'Trailblazer',
  'payload_count': 2},
 {'_id': ObjectId('69beeada1eb88c8f9b0d78fb'),
  'core_count': 1,
  'name': 'RatSat',
  'payload_count': 1},
 {'_id': ObjectId('69beeada1eb88c8f9b0d78fc'),
  'core_count': 1,
  'name': 'RazakSat',
  'payload_count': 1},
 {'_id': ObjectId('69beeada1eb88c8f9b0d78fd'),
  'core_count': 1,
  'name': 'Falcon 9 Test Flight',
  'payload_count': 1},
 {'_id': ObjectId('69beeada1eb88c8f9b0d78fe'),
  'core_count': 1,
  'name': 'COTS 1',
  'payload_count': 2},
 {'_id': ObjectId('69beeada1eb88c8f9b0d78ff'),
  'core_count': 1,
  'name': 'COTS 2',
  'payload_count': 1},
 {'_id': ObjectId('69beeada1eb88c8f9b0d7900'),
  'core_count': 1,
  'name': 'CRS-1',
  'payload_count': 2},
 {'_id': ObjectId('69beeada1eb88c8f9b0d7901'),
  'core_count': 1,
```

```
  'name': 'CRS-2',
  'payload_count': 1}]
```

Most launches carry a single core and one or two payloads. This is expected since the majority of SpaceX missions use a single Falcon 9 booster. Falcon Heavy missions, which use three cores, would show a higher `core_count`. The `cores` array contains embedded documents with landing information, making it a key source for the reuse and landing analyses later.

```python
# check launches empty cores and payloads
print("Launches with empty/missing cores:",
      launches_raw.count_documents({
          "$expr": {"$eq": [{"$size": {"$ifNull": ["$cores", []]}}, 0]}
      }))

print("Launches with empty/missing payloads:",
      launches_raw.count_documents({
          "$expr": {"$eq": [{"$size": {"$ifNull": ["$payloads", []]}}, 0]}
      }))
```

```
Launches with empty/missing cores: 0
Launches with empty/missing payloads: 12
```

No launches have an empty `cores` array, meaning every launch has at least one booster entry — this is essential for the core reuse and landing analyses. However, 12 launches have empty `payloads` arrays. This may be because these 12 empty payload launches were test flights. Since payload data is not central to the research questions of this project, no further investigation was done and these entries do not need to be excluded.

```python
# inspect launchpads: check status distribution and launch attempt counts
pp.pprint(list(launchpads_raw.aggregate([
    {
        "$project": {
            "_id": 0,
            "name": 1,
            "full_name": 1,
            "status": 1,
            "launch_attempts": 1,
            "launch_successes": 1,
            "region": 1
        }
    },
    {"$sort": {"launch_attempts": -1}}
])))
```

```
[{'full_name': 'Cape Canaveral Space Force Station Space Launch Complex 40',
  'launch_attempts': 99,
  'launch_successes': 97,
```

```
  'name': 'CCSFS SLC 40',
  'region': 'Florida',
  'status': 'active'},
 {'full_name': 'Kennedy Space Center Historic Launch Complex 39A',
  'launch_attempts': 55,
  'launch_successes': 55,
  'name': 'KSC LC 39A',
  'region': 'Florida',
  'status': 'active'},
 {'full_name': 'Vandenberg Space Force Base Space Launch Complex 4E',
  'launch_attempts': 28,
  'launch_successes': 27,
  'name': 'VAFB SLC 4E',
  'region': 'California',
  'status': 'active'},
 {'full_name': 'Kwajalein Atoll Omelek Island',
  'launch_attempts': 5,
  'launch_successes': 2,
  'name': 'Kwajalein Atoll',
  'region': 'Marshall Islands',
  'status': 'retired'},
 {'full_name': 'Vandenberg Space Force Base Space Launch Complex 3W',
  'launch_attempts': 0,
  'launch_successes': 0,
  'name': 'VAFB SLC 3W',
  'region': 'California',
  'status': 'retired'},
 {'full_name': 'SpaceX South Texas Launch Site',
  'launch_attempts': 0,
  'launch_successes': 0,
  'name': 'STLS',
  'region': 'Texas',
  'status': 'under construction'}]
```

The launchpads collection contains 6 entries spanning three statuses: active, retired, and under construction. CCSFS SLC 40 leads with 99 launch attempts, followed by KSC LC 39A with 55 and VAFB SLC 4E with 28. KSC LC 39A stands out with a perfect 55/55 success record. Kwajalein Atoll is the only non-US-mainland pad, located in the Marshall Islands, and was used exclusively for early Falcon 1 missions with only 2 out of 5 successes. VAFB SLC 3W is retired with zero recorded attempts, and STLS (SpaceX South Texas Launch Site) is still under construction. All documents contain the expected fields without null values.

```python
# inspect landingpads: check type distribution, status and landing attempt counts
pp.pprint(list(landingpads_raw.aggregate([
    {
        "$project": {
            "_id": 0,
            "name": 1,
```

```
            "full_name": 1,
            "type": 1,
            "status": 1,
            "landing_attempts": 1,
            "landing_successes": 1,
            "region": 1
        }
    },
    {"$sort": {"landing_attempts": -1}}
]))
```

```
[{'full_name': 'Of Course I Still Love You',
  'landing_attempts': 61,
  'landing_successes': 54,
  'name': 'OCISLY',
  'region': 'California',
  'status': 'active',
  'type': 'ASDS'},
 {'full_name': 'Just Read The Instructions',
  'landing_attempts': 40,
  'landing_successes': 39,
  'name': 'JRTI',
  'region': 'Florida',
  'status': 'active',
  'type': 'ASDS'},
 {'full_name': 'Landing Zone 1',
  'landing_attempts': 21,
  'landing_successes': 20,
  'name': 'LZ-1',
  'region': 'Florida',
  'status': 'active',
  'type': 'RTLS'},
 {'full_name': 'A Shortfall of Gravitas',
  'landing_attempts': 21,
  'landing_successes': 21,
  'name': 'ASOG',
  'region': 'Florida',
  'status': 'active',
  'type': 'ASDS'},
 {'full_name': 'Landing Zone 4',
  'landing_attempts': 6,
  'landing_successes': 6,
  'name': 'LZ-4',
  'region': 'California',
  'status': 'active',
  'type': 'RTLS'},
 {'full_name': 'Landing Zone 2',
  'landing_attempts': 3,
```

```
  'landing_successes': 3,
  'name': 'LZ-2',
  'region': 'Florida',
  'status': 'active',
  'type': 'RTLS'},
 {'full_name': 'Just Read The Instructions V1',
  'landing_attempts': 2,
  'landing_successes': 0,
  'name': 'JRTI-1',
  'region': 'Florida',
  'status': 'retired',
  'type': 'ASDS'}]
```

The landingpads collection contains 7 entries with two distinct types: ASDS (Autonomous Spaceport Drone Ship) for sea-based recovery and RTLS (Return to Launch Site) for land-based recovery. Four pads are ASDS and three are RTLS. OCISLY (Of Course I Still Love You) leads with 61 landing attempts but only 54 successes, while ASOG (A Shortfall of Gravitas) shows a perfect 21/21 record. The three RTLS pads (LZ-1, LZ-2, LZ-4) generally show higher success rates than the drone ships, which is expected given the more controlled conditions of land-based landings. JRTI-1 is the only retired pad with 0 out of 2 successful landings, reflecting the early experimental phase of drone ship recovery. Note that the `landing_attempts` and `landing_successes` fields are pre-aggregated by the API and may include the 19 stale launches, so the analysis pipelines will recompute these values from the staged launch data instead.

```python
# check cores with null/missing landpad
result = list(launches_raw.aggregate([
    {"$unwind": "$cores"},
    {"$match": {"cores.landpad": None}},
    {
        "$project": {
            "_id": 0,
            "name": 1,
            "date_utc": 1,
            "core_id": "$cores.core",
            "flight": "$cores.flight",
            "landing_attempt": "$cores.landing_attempt",
            "landing_success": "$cores.landing_success",
            "landpad": "$cores.landpad"
        }
    },
    {"$sort": {"date_utc": 1}}
]))
print(f"Cores with null/missing landpad: {len(result)}")
pp.pprint(result[:10])
```

```
Cores with null/missing landpad: 58
[{'core_id': '5e9e289df35918033d3b2623',
  'date_utc': '2006-03-24T22:30:00.000Z',
```

```
 'flight': 1,
 'landing_attempt': False,
 'landing_success': None,
 'landpad': None,
 'name': 'FalconSat'},
{'core_id': '5e9e289ef35918416a3b2624',
 'date_utc': '2007-03-21T01:10:00.000Z',
 'flight': 1,
 'landing_attempt': False,
 'landing_success': None,
 'landpad': None,
 'name': 'DemoSat'},
{'core_id': '5e9e289ef3591814873b2625',
 'date_utc': '2008-08-03T03:34:00.000Z',
 'flight': 1,
 'landing_attempt': False,
 'landing_success': None,
 'landpad': None,
 'name': 'Trailblazer'},
{'core_id': '5e9e289ef3591855dc3b2626',
 'date_utc': '2008-09-28T23:15:00.000Z',
 'flight': 1,
 'landing_attempt': False,
 'landing_success': None,
 'landpad': None,
 'name': 'RatSat'},
{'core_id': '5e9e289ef359184f103b2627',
 'date_utc': '2009-07-13T03:35:00.000Z',
 'flight': 1,
 'landing_attempt': False,
 'landing_success': None,
 'landpad': None,
 'name': 'RazakSat'},
{'core_id': '5e9e289ef359185f2b3b2628',
 'date_utc': '2010-06-04T18:45:00.000Z',
 'flight': 1,
 'landing_attempt': False,
 'landing_success': None,
 'landpad': None,
 'name': 'Falcon 9 Test Flight'},
{'core_id': '5e9e289ef35918187c3b2629',
 'date_utc': '2010-12-08T15:43:00.000Z',
 'flight': 1,
 'landing_attempt': False,
 'landing_success': None,
 'landpad': None,
 'name': 'COTS 1'},
{'core_id': '5e9e289ef35918f39c3b262a',
 'date_utc': '2012-05-22T07:44:00.000Z',
```

```
  'flight': 1,
  'landing_attempt': False,
  'landing_success': None,
  'landpad': None,
  'name': 'COTS 2'},
 {'core_id': '5e9e289ff3591821a73b262b',
  'date_utc': '2012-10-08T00:35:00.000Z',
  'flight': 1,
  'landing_attempt': False,
  'landing_success': None,
  'landpad': None,
  'name': 'CRS-1'},
 {'core_id': '5e9e289ff3591884e03b262c',
  'date_utc': '2013-03-01T19:10:00.000Z',
  'flight': 1,
  'landing_attempt': False,
  'landing_success': None,
  'landpad': None,
  'name': 'CRS-2'}]
```

```python
# check cores with more than 1 landpad
result = list(launches_raw.aggregate([
    {"$unwind": "$cores"},
    {"$match": {"cores.landpad": {"$type": "array"}}},
    {
        "$project": {
            "_id": 0,
            "name": 1,
            "core_id": "$cores.core",
            "landpad": "$cores.landpad"
        }
    }
]))
print(f"Cores with more than 1 landpad: {len(result)}")
pp.pprint(result)
```

```
Cores with more than 1 landpad: 0
[]
```

4.4 Data Cleaning & Transformation

With the insights from the inspection, the data can now be transformed and then stored in the staging collections. The key issues to address are: converting string IDs to MongoDB `_id` fields, transforming date strings into BSON Date objects, filtering out the 19 unreliable launches, and projecting only the fields relevant to the analysis. Before that, the collections can be deleted in order to ensure no duplicate entries, similar to the dropping of raw data collections.

```python
# OPTIONAL: delete stage data
launches_stage.drop()
launchpads_stage.drop()
rockets_stage.drop()
landingpads_stage.drop()

print("launches_stage:", launches_stage.count_documents({}))
print("rockets_stage:", rockets_stage.count_documents({}))
print("launchpads_stage:", launchpads_stage.count_documents({}))
print("landinpads_stage:", landingpads_stage.count_documents({}))
```

```
launches_stage: 0
rockets_stage: 0
launchpads_stage: 0
landinpads_stage: 0
```

The rockets transformation addresses the need to standardize identifiers by remapping the API's `id` field to MongoDB's `_id`. Additionally, the `first_flight` string is converted to a Date object using `$dateFromString`, consistent with the date format issue identified during inspection. Only fields relevant to the analysis — such as `name`, `cost_per_launch`, and `success_rate_pct` — are retained via `$project` to keep the staging collection lean.

```python
rockets_stage = db["rockets_stage"]

# rename id to _id and convert first_flight string to a date
set_fields = {"$set": {
    "_id": "$id",
    "first_flight_date": {"$dateFromString": {"dateString": "$first_flight"}}
}}

# keep only relevant fields
project = {"$project": {
    "_id": 1,
    "name": 1,
    "cost_per_launch": 1,
    "success_rate_pct": 1,
    "country": 1,
    "company": 1,
    "first_flight_date": 1
}}

# write results to rockets_stage collection
merge = {"$merge": "rockets_stage"}
```

```python
pipeline = [set_fields, project, merge]
list(rockets_raw.aggregate(pipeline))
print(rockets_stage.count_documents({}))
```

4

Similar to rockets, the launchpad `id` is remapped to `_id`. A new `location` field is derived by concatenating `locality` and `region` using `$concat`, providing a human-readable location string for use in visualizations. Geographic coordinates (`latitude`, `longitude`) and launch statistics are preserved for potential spatial analysis.

```
launchpads_stage = db["launchpads_stage"]

# rename id to _id and concatenate locality and region into a location string
set_fields = {"$set": {
    "_id": "$id",
    "location": {"$concat": ["$locality", ", ", "$region"]}
}}

# keep only relevant fields
project = {"$project": {
    "_id": 1,
    "name": 1,
    "full_name": 1,
    "locality": 1,
    "region": 1,
    "latitude": 1,
    "longitude": 1,
    "launch_attempts": 1,
    "launch_successes": 1,
    "location": 1,
    "status": 1
}}

# write results to launchpads_stage collection
merge = {"$merge": "launchpads_stage"}
```

```
pipeline = [set_fields, project, merge]
list(launchpads_raw.aggregate(pipeline))
print(launchpads_stage.count_documents({}))
```

6

The launches transformation is the most involved, directly addressing several issues found during inspection. The `$match` stage at the beginning filters out the 19 launches with `success: null`, reducing the dataset from 205 to 186 documents. The `date_utc` string is converted to a proper Date object, resolving the type mismatch identified earlier. A `launch_year` field is derived using `$year` for time-based grouping, and the sizes of the `cores` and `payloads` arrays are precomputed as `core_count` and `payload_count`, avoiding repeated `$size` calculations in downstream pipelines.

```
launches_stage = db["launches_stage"]

# filter out launches where success is null (abandoned API data from mid-2022 onwards)
match = {"$match": {"success": {"$ne": None}}}

# rename id to _id and convert date_utc string to a date object
set_date = {"$set": {
    "_id": "$id",
    "date": {"$dateFromString": {"dateString": "$date_utc"}}
}}

# extract launch year and compute array sizes for cores and payloads
set_derived = {"$set": {
    "launch_year": {"$year": "$date"},
    "core_count": {"$size": {"$ifNull": ["$cores", []]}},
    "payload_count": {"$size": {"$ifNull": ["$payloads", []]}}
}}

# keep only relevant fields
project = {"$project": {
    "_id": 1,
    "name": 1,
    "date": 1,
    "launch_year": 1,
    "success": 1,
    "rocket": 1,
    "launchpad": 1,
    "cores": 1,
    "payloads": 1,
    "core_count": 1,
    "payload_count": 1
}}

# write results to launches_stage collection
merge = {"$merge": "launches_stage"}
```

```
pipeline = [match, set_date, set_derived, project, merge]
list(launches_raw.aggregate(pipeline))
print(launches_stage.count_documents({}))
```

```
186
```

The landing pads follow the same pattern as launchpads: `id` is remapped to `_id`, and a `location` string is concatenated from `locality` and `region`. The `type` field (ASDS or RTLS) is preserved, as it will be used to compare landing success rates between autonomous drone ships and return-to-launch-site pads in the analysis.

```
landingpads_stage = db["landingpads_stage"]

# rename id to _id and concatenate locality and region into a location string
set_fields = {"$set": {
    "_id": "$id",
    "location": {"$concat": ["$locality", ", ", "$region"]}
}}

# keep only relevant fields
project = {"$project": {
    "_id": 1,
    "name": 1,
    "full_name": 1,
    "type": 1,
    "locality": 1,
    "region": 1,
    "latitude": 1,
    "longitude": 1,
    "landing_attempts": 1,
    "landing_successes": 1,
    "location": 1,
    "status": 1
}}

# write results to landingpads_stage collection
merge = {"$merge": "landingpads_stage"}
```

58 cores have a null `landpad` field. As the first 10 results show, these are all early missions (2006–2013) where `landing_attempt` is `false` and no recovery was attempted. This is expected, as SpaceX did not begin attempting booster landings until late 2015. These cores are not erroneous and do not need to be cleaned — they simply represent missions where landing infrastructure was not yet available. The analysis pipelines in sections 6.3 and 6.6 already handle this by filtering on `landing_attempt: true`.

```
pipeline = [set_fields, project, merge]
list(landingpads_raw.aggregate(pipeline))
print(landingpads_stage.count_documents({}))
```

7

No cores store `landpad` as an array, confirming that the field is always either a single string reference or null. This means the relationship between a core entry and a landing pad is `0..1` — each core references at most one landing pad. This finding is reflected in the class diagram and ensures that no special handling for multi-valued `landpad` fields is needed in the analysis pipelines.

5 Class Diagramm

The class diagram below visualizes the structure of the staged data as it is stored in MongoDB. It shows the four collections used in this project, their fields and data types, as well as the relationships between them.

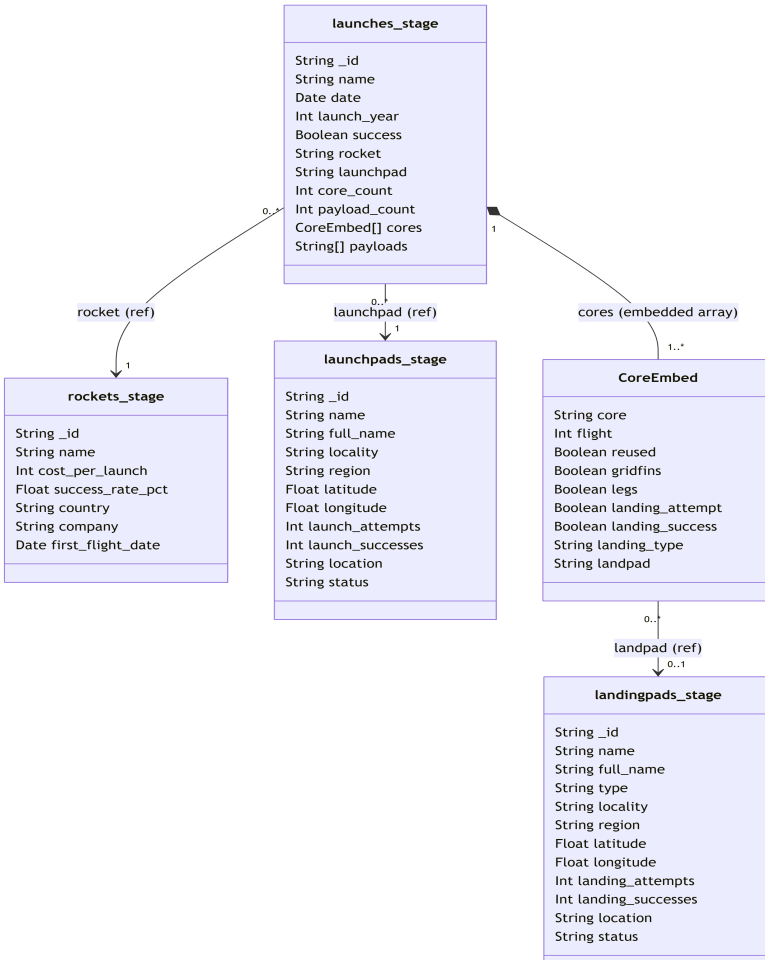

Figure 1: Class Diagram of Staged SpaceX Data

The central collection is `launches_stage`, which holds one document per launch. Each launch references exactly one rocket from `rockets_stage` and one launchpad from `launchpads_stage` via string ID fields. These are not embedded documents but foreign key-style references, which require `$lookup` stages during analysis to join the data.

Every launch contains an embedded array of `CoreEmbed` objects representing the boosters used in the mission. The `1..*` multiplicity confirms that no launch exists without at least one core. Most launches have a single core (Falcon 9), while Falcon Heavy missions carry three. Each `CoreEmbed` optionally references a landing pad from `landingpads_stage` via the `landpad` field. The `0..1` multiplicity reflects

that 58 cores have no landing pad assigned — these correspond to early missions where booster recovery was not yet attempted. The composition relationship between `launches_stage` and `CoreEmbed` indicates that cores do not exist independently but are always embedded within a launch document

6 Analysis

6.1 Launch success rate by rocket

This pipeline begins by filtering out any launches that lack a definitive outcome (success is either `True` or `False`). It then performs a `$lookup` to join each launch with its corresponding rocket document from `rockets_stage`, using the string ID stored in the rocket field. After flattening the joined array with `$unwind`, the pipeline groups all launches by rocket name and counts total launches, successes, and failures. A `$project` stage computes the success rate as a percentage (rounded to one decimal), and the results are sorted in descending order by total launches. This pipeline demonstrates the use of `$lookup` for cross-collection joins, `$cond` for conditional counting, and arithmetic operators (`$divide`, `$multiply`, `$round`) for derived metrics.

```
# exclude upcoming launches without a result
match = {"$match": {"success": {"$in": [True, False]}}}

# join rocket details from rockets_stage by rocket id
lookup = {"$lookup": {
    "from": "rockets_stage",
    "localField": "rocket",
    "foreignField": "_id",
    "as": "rocket_info"
}}

# flatten rocket_info array into a single object
unwind = {"$unwind": "$rocket_info"}

# group by rocket name and count launches, successes, failures
group = {"$group": {
    "_id": "$rocket_info.name",
    "total_launches": {"$sum": 1},
    "successful": {"$sum": {"$cond": ["$success", 1, 0]}},
    "failed": {"$sum": {"$cond": ["$success", 0, 1]}},
    "cost_per_launch": {"$first": "$rocket_info.cost_per_launch"}
}}

# calculate success rate and clean up output fields
project = {"$project": {
    "_id": 0,
    "rocket": "$_id",
    "total_launches": 1,
    "successful": 1,
    "failed": 1,
```

```
    "success_rate_pct": {"$round": [
        {"$multiply": [{"$divide": ["$successful", "$total_launches"]}, 100]},
        1
    ]},
    "cost_per_launch": 1
}}

# sort by most launched rocket first
sort = {"$sort": {"total_launches": -1}}
```

```
pipeline = [match, lookup, unwind, group, project, sort]
cursor = launches_stage.aggregate(pipeline)
pd.DataFrame(cursor)
```

Table 1: Launch Success Rate by Rocket

	total_launches	successful	failed	cost_per_launch	rocket	success_rate_pct
0	178	176	2	50000000	Falcon 9	98.9
1	5	2	3	6700000	Falcon 1	40.0
2	3	3	0	90000000	Falcon Heavy	100.0

```
pipeline = [match, lookup, unwind, group, project, sort]
results = list(launches_stage.aggregate(pipeline))

rockets = [r["rocket"] for r in results]
success_rates = [r["success_rate_pct"] for r in results]
total_launches = [r["total_launches"] for r in results]

fig, ax = plt.subplots(figsize=(8, 3))
bars = ax.barh(rockets, success_rates, color=["#2ecc71" if r >= 90 else "#e67e22" if r >= 70 else "#e

for bar, total, rate in zip(bars, total_launches, success_rates):
    ax.text(bar.get_width() - 1, bar.get_y() + bar.get_height() / 2,
            f"{rate}% ({total} launches)", va="center", ha="right", fontsize=9, color="white", fontwe

ax.set_xlabel("Success Rate (%)")
ax.set_xlim(0, 100)
ax.invert_yaxis()
plt.tight_layout()
plt.savefig("Personal_project_files/images/success_by_rocket.png", dpi=150)
plt.show()
```

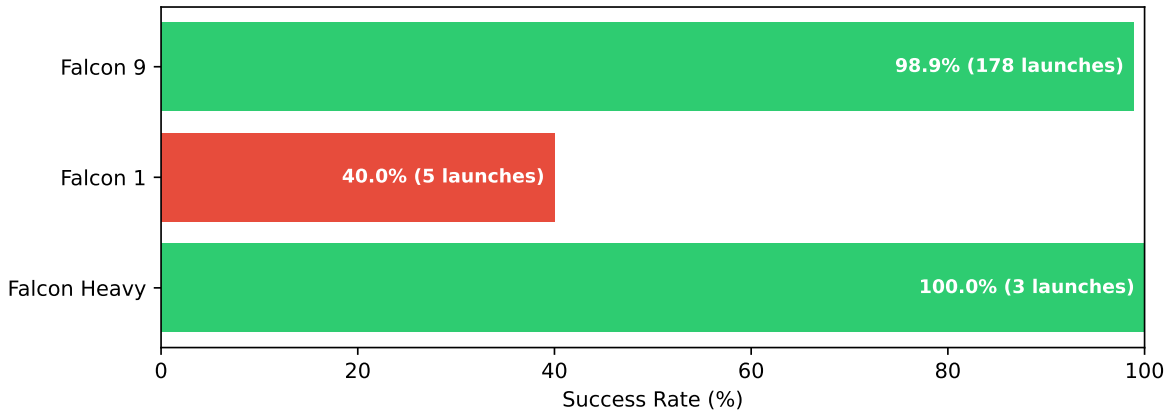

Figure 2: Launch Success Rate by Rocket

The horizontal bar chart reveals that Falcon 9 dominates SpaceX's launch history with 178 missions and an exceptional 98.9% success rate. The rocket has only 2 failures across its entire operational life. Falcon Heavy, while still in its early phase with just 3 launches, holds a perfect 100% record. Falcon 1, SpaceX's first orbital rocket, succeeded only 2 out of 5 times, reflecting the steep learning curve of the company's early years. The colour coding, green for 90%, orange for 70%, red below, makes it immediately clear that SpaceX's current fleet is extraordinarily reliable.

6.2 Most active launchpads

Unlike the previous pipeline, this one starts from the `launchpads_stage` collection to ensure that even launchpads with zero launches appear in the output. A `$lookup` brings in all matching launches, and `$addFields` computes three derived values: the total number of launches, the number of successful launches using `$filter` and `$size`, and the set of distinct active years. A second `$addFields` stage calculates the success rate with a guard against division by zero and counts the years span. The final `$project` and `$sort` stages clean up the output and rank launchpads by activity. This pipeline showcases `$filter` for array-level conditional logic and `$setUnion` for extracting unique values from nested arrays.

```
# start from launchpads so all launchpads are always included
match_active = {"$match": {"_id": {"$exists": True}}}

# lookup all launches for each launchpad
lookup = {"$lookup": {
    "from": "launches_stage",
    "localField": "_id",
    "foreignField": "launchpad",
    "as": "launches_data"
}}

# compute stats from the joined launches array
add_fields = {"$addFields": {
```

```
    "launches": {"$size": "$launches_data"},
    "successful": {
        "$size": {
            "$filter": {
                "input": "$launches_data",
                "as": "l",
                "cond": {"$eq": ["$$l.success", True]}
            }
        }
    },
    "years_active": {
        "$setUnion": [
            "$launches_data.launch_year"
        ]
    }
}}

# compute success rate and years span
add_fields2 = {"$addFields": {
    "success_rate": {
        "$cond": [
            {"$gt": ["$launches", 0]},
            {"$round": [
                {"$multiply": [
                    {"$divide": ["$successful", "$launches"]},
                    100
                ]},
                1
            ]},
            0  # avoid division by zero for pads with 0 launches
        ]
    },
    "years_span": {"$size": "$years_active"}
}}

# clean up output fields
project = {"$project": {
    "_id": 0,
    "launchpad": "$name",
    "status": 1,
    "launches": 1,
    "successful": 1,
    "success_rate": 1,
    "years_span": 1
}}

# sort by most active launchpad first
sort = {"$sort": {"launches": -1}}
```

```python
pipeline = [match_active, lookup, add_fields, add_fields2, project, sort]
cursor = launchpads_stage.aggregate(pipeline)
pd.DataFrame(cursor)
```

Table 2: Most Active SpaceX Launchpads

	status	launches	successful	success_rate	years_span	launchpad
0	active	99	97	98.0	12	CCSFS SLC 40
1	active	55	55	100.0	6	KSC LC 39A
2	active	27	27	100.0	8	VAFB SLC 4E
3	retired	5	2	40.0	4	Kwajalein Atoll
4	retired	0	0	0.0	0	VAFB SLC 3W
5	under construction	0	0	0.0	0	STLS

```python
pipeline = [match_active, lookup, add_fields, add_fields2, project, sort]
results = list(launchpads_stage.aggregate(pipeline))

names = [r["launchpad"] for r in results]
launches = [r["launches"] for r in results]
successful = [r["successful"] for r in results]
failed = [r["launches"] - r["successful"] for r in results]

fig, ax = plt.subplots(figsize=(8, 4))

# stacked bar: successful + failed
bars_success = ax.barh(names, successful, color="#2ecc71", label="Successful")
bars_failed = ax.barh(names, failed, left=successful, color="#e74c3c", label="Failed")

# labels inside bars showing total launches
for i, (name, total, rate) in enumerate(zip(names, launches, [r["success_rate"] for r in results])):
    if total > 0:
        ax.text(total + 0.5, i, f"{total} launches ({rate}%)",
                va="center", ha="left", fontsize=9)

ax.set_xlabel("Number of Launches")
ax.set_xlim(0, max(launches) + 40)
ax.invert_yaxis()
ax.legend(loc="lower right")
plt.tight_layout()
plt.show()
```

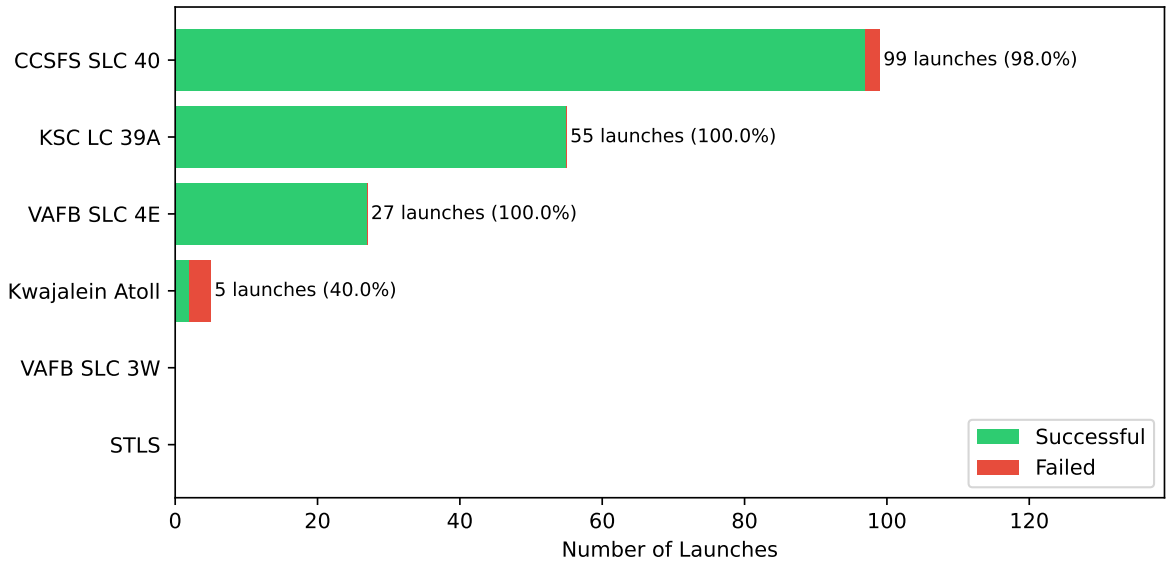

Figure 3: Most Active SpaceX Launchpads

The stacked bar chart shows that CCSFS SLC 40 in Florida is SpaceX's workhorse facility with 99 launches and a 98% success rate. Its 2 failures are the only blemishes across all active US-mainland pads. KSC LC 39A and VAFB SLC 4E both maintain flawless 100% records with 55 and 27 launches respectively. Kwajalein Atoll stands out as the sole non-continental site with a low 40% success rate across 5 early Falcon 1 missions. VAFB SLC 3W (retired, zero launches) and STLS (under construction) confirm that SpaceX's operational launch capacity is currently concentrated across three active Florida and California sites.

6.3 Landing success rate by landing pad type

This pipeline starts from landingpads_stage and uses an advanced `$lookup` with a sub-pipeline: it unwinds the cores array inside each launch document and matches only those core entries whose landpad field equals the current landing pad's `_id`. This approach directly links individual booster landing events to their target pads. Two `$addFields` stages then compute landing attempts, successes, and the success rate, again with a guard against division by zero. The output is projected with pad name, `type` (ASDS or RTLS), `location`, and `status`, then sorted by attempts. This pipeline is notable for its use of the correlated sub-pipeline form of `$lookup` with let and `$expr`, which enables filtering inside the joined collection before results are returned.

```
# start from landingpads so all pads are always included
match_all = {"$match": {"_id": {"$exists": True}}}

# lookup all core entries that reference this landingpad
lookup = {"$lookup": {
    "from": "launches_stage",
    "let": {"pad_id": "$_id"},
```

```
    "pipeline": [
        {"$unwind": "$cores"},
        {"$match": {"$expr": {"$eq": ["$cores.landpad", "$$pad_id"]}}}
    ],
    "as": "core_landings"
}}

# compute landing attempts and successes from matched core entries
add_fields = {"$addFields": {
    "landing_attempts": {"$size": "$core_landings"},
    "landing_successes": {
        "$size": {
            "$filter": {
                "input": "$core_landings",
                "as": "c",
                "cond": {"$eq": ["$$c.cores.landing_success", True]}
            }
        }
    }
}}

# compute landing success rate, guard against division by zero
add_fields2 = {"$addFields": {
    "landing_success_rate": {
        "$cond": [
            {"$gt": ["$landing_attempts", 0]},
            {"$round": [
                {"$multiply": [
                    {"$divide": ["$landing_successes", "$landing_attempts"]},
                    100
                ]},
                1
            ]},
            None
        ]
    }
}}

# clean up output fields
project = {"$project": {
    "_id": 0,
    "landingpad": "$name",
    "type": 1,
    "status": 1,
    "landing_attempts": 1,
    "landing_successes": 1,
    "landing_success_rate": 1
}}
```

```python

# sort by most attempted landingpad first
sort = {"$sort": {"landing_attempts": -1}}
```

```python
pipeline = [match_all, lookup, add_fields, add_fields2, project, sort]
cursor = landingpads_stage.aggregate(pipeline)
pd.DataFrame(cursor)
```

Table 3: Landing success Rate per Landing Pad

	status	type	landing_attempts	landing_successes	landing_success_rate	landingpad
0	active	ASDS	63	54	85.7	OCISLY
1	active	ASDS	40	39	97.5	JRTI
2	active	RTLS	21	20	95.2	LZ-1
3	active	ASDS	21	21	100.0	ASOG
4	active	RTLS	6	6	100.0	LZ-4
5	active	RTLS	3	3	100.0	LZ-2
6	retired	ASDS	2	0	0.0	JRTI-1

```python
pipeline = [match_all, lookup, add_fields, add_fields2, project, sort]
results = [r for r in landingpads_stage.aggregate(pipeline) if r["landing_attempts"] > 0]

names = [r["landingpad"] for r in results]
names = [n.replace("-", "-") for n in names]
success_rates = [r["landing_success_rate"] for r in results]
attempts = [r["landing_attempts"] for r in results]

fig, ax = plt.subplots(figsize=(8, 4))
bars = ax.barh(names, success_rates, color=["#2ecc71" if r >= 90 else "#e67e22" if r >= 70 else "#e74

for bar, attempt, rate in zip(bars, attempts, success_rates):
    ax.text(bar.get_width() - 1, bar.get_y() + bar.get_height() / 2,
            f"{rate}% ({attempt} attempts)", va="center", ha="right",
            fontsize=9, color="white", fontweight="bold")

ax.set_ylabel("Landing Pad")
ax.set_xlabel("Landing Success Rate (%)")
ax.set_xlim(0, 100)
ax.invert_yaxis()
plt.tight_layout()
plt.savefig("Personal_project_files/images/landing_success_per_pad.png", dpi=150)
plt.show()
```

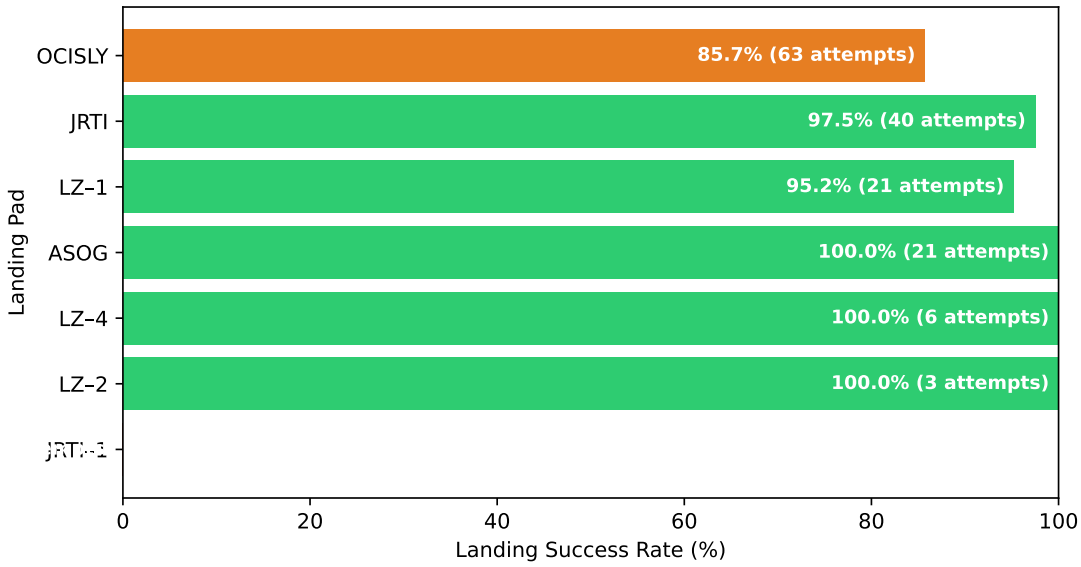

Figure 4: Landing Success Rate per Landing Pad

The chart confirms an intuitive pattern: land-based recovery (RTLS) is more reliable than sea-based recovery (ASDS). LZ-2, LZ-4, and ASOG all achieve a perfect 100% landing rate, while LZ-1 is close behind at 95.2%. Among the drone ships, JRTI performs best at 97.5%, whereas OCISLY, despite being the most-used pad with 63 attempts, has the lowest active-pad success rate at 85.7%, accounting for the majority of SpaceX's landing failures. The retired JRTI-1, with 0 out of 2 successes, reflects the experimental early days of autonomous ship landings. Overall, SpaceX's landing infrastructure has matured to a remarkably high standard across both platforms.

6.4 Launch frequency over time

This pipeline groups all valid launches by their pre-computed `launch_year` field, counting totals, successes, and failures per year using `$group` with conditional `$sum` expressions. An `$addFields` stage derives the annual success rate, and the results are sorted chronologically. A final `$project` renames `_id` to year for readability. Although structurally simpler than the cross-collection pipelines, this aggregation is effective for time-series analysis and demonstrates how pre-computed fields simplify downstream aggregations.

```
# exclude upcoming launches without a result
match = {"$match": {"success": {"$in": [True, False]}}}

# group by year and count total, successful and failed launches
group = {"$group": {
    "_id": "$launch_year",
    "total": {"$sum": 1},
    "successful": {"$sum": {"$cond": ["$success", 1, 0]}},
    "failed": {"$sum": {"$cond": ["$success", 0, 1]}}
```

```
}}

# compute success rate per year
add_fields = {"$addFields": {
    "success_rate_pct": {"$round": [
        {"$multiply": [{"$divide": ["$successful", "$total"]}, 100]},
        1
    ]}
}}

# sort chronologically
sort = {"$sort": {"_id": 1}}

# rename _id to year and clean up output fields
project = {"$project": {
    "_id": 0,
    "year": "$_id",
    "total": 1,
    "successful": 1,
    "failed": 1,
    "success_rate_pct": 1
}}
```

```
pipeline = [match, group, add_fields, sort, project]
cursor = launches_stage.aggregate(pipeline)
pd.DataFrame(cursor)
```

Table 4: Launch Frequency and Success Rate Over Time

	total	successful	failed	success_rate_pct	year
0	1	0	1	0.0	2006
1	1	0	1	0.0	2007
2	2	1	1	50.0	2008
3	1	1	0	100.0	2009
4	2	2	0	100.0	2010
5	2	2	0	100.0	2012
6	3	3	0	100.0	2013
7	6	6	0	100.0	2014
8	7	6	1	85.7	2015
9	9	8	1	88.9	2016
10	18	18	0	100.0	2017
11	21	21	0	100.0	2018
12	13	13	0	100.0	2019
13	26	26	0	100.0	2020
14	31	31	0	100.0	2021
15	43	43	0	100.0	2022

```python
pipeline = [match, group, add_fields, sort, project]
results = list(launches_stage.aggregate(pipeline))

years = [r["year"] for r in results]
totals = [r["total"] for r in results]
success_rates = [r["success_rate_pct"] for r in results]

fig, ax1 = plt.subplots(figsize=(10, 4))

# bar chart for total launches
ax1.bar(years, totals, color="#3498db", alpha=0.7, label="Total Launches")
ax1.set_xlabel("Year")
ax1.set_xticks(range(min(years), max(years) + 1))
ax1.set_xticklabels([str(y) for y in range(min(years), max(years) + 1)], rotation=45, ha="right")
ax1.set_ylabel("Number of Launches", color="#3498db")
ax1.tick_params(axis="y", labelcolor="#3498db")

# line chart for success rate on second y-axis
ax2 = ax1.twinx()
ax2.plot(years, success_rates, color="#e74c3c", marker="o", linewidth=2, label="Success Rate %")
ax2.set_ylabel("Success Rate (%)", color="#e74c3c")
ax2.tick_params(axis="y", labelcolor="#e74c3c")
ax2.set_ylim(0, 110)

fig.legend(loc="upper left", bbox_to_anchor=(0.1, 0.95))
plt.tight_layout()
plt.savefig("Personal_project_files/images/launch_frequency.png", dpi=150)
plt.show()
```

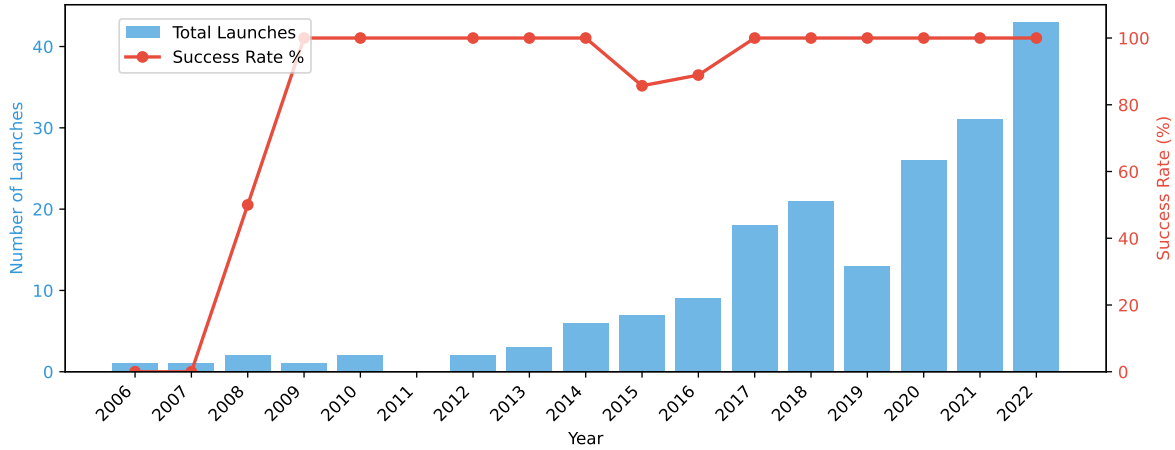

Figure 5: Launch Frequency and Success Rate Over Time

The dual-axis chart tells a compelling growth story. Launch volume increased slowly from 1–2 missions

per year to 43 in 2022, showing near-exponential growth from 2017 onward. The success rate line mirrors this maturation: early years were volatile (0% in 2006–2007, 50% in 2008), but from 2017 onward SpaceX achieved a perfect or near-perfect success rate every year. The two dips in 2015 and 2016 correspond to the CRS-7 in-flight breakup and the Amos-6 pad explosion respectively — both pivotal incidents that led to design improvements. The combination of rising cadence and sustained reliability from 2017 onward underscores SpaceX's transition from an experimental startup to a mature, high-tempo launch provider.

6.5 Core reuse analysis

This pipeline first unwinds the cores array so each booster entry becomes its own document, then filters out entries with missing core IDs or flight numbers. A `$group` stage aggregates per core, taking the maximum flight number as the total flights for that booster and summing landing attempts and successes. The key feature is the `$bucket` stage, which bins cores into flight-count ranges (1, 2, 3–4, 5–7, 8–19), counting how many cores fall into each bucket and computing the average number of successful landings per bucket. This pipeline demonstrates `$bucket` for histogram-style analysis and `$addToSet` for collecting distinct values within groups.

```
# unwind cores array so each core entry becomes its own document
unwind = {"$unwind": "$cores"}

# exclude entries where core id or flight number is missing
match = {"$match": {
    "cores.core": {"$ne": None},
    "cores.flight": {"$ne": None}
}}

# group by core id and collect flight and landing stats
group = {"$group": {
    "_id": "$cores.core",
    "total_flights": {"$max": "$cores.flight"},
    "landing_attempts": {"$sum": {"$cond": [
        {"$eq": ["$cores.landing_attempt", True]}, 1, 0
    ]}},
    "landing_successes": {"$sum": {"$cond": [
        {"$eq": ["$cores.landing_success", True]}, 1, 0
    ]}},
    "landing_types": {"$addToSet": "$cores.landing_type"}
}}

# bucket cores by number of total flights to see reuse distribution
bucket = {"$bucket": {
    "groupBy": "$total_flights",
    "boundaries": [1, 2, 3, 5, 8, 20],
    "default": "20+",
    "output": {
        "core_count": {"$sum": 1},
        "avg_landing_success": {"$avg": "$landing_successes"}
```

```
    }
}}

# rename _id to flight_range and round average landing successes
project = {"$project": {
    "_id": 0,
    "flight_range": "$_id",
    "core_count": 1,
    "avg_landing_success": {"$round": ["$avg_landing_success", 1]}
}}
```

```
pipeline = [unwind, match, group, bucket, project]
cursor = launches_stage.aggregate(pipeline)
pd.DataFrame(cursor)
```

Table 5: Core Reuse Distribution and Landing Success

	core_count	flight_range	avg_landing_success
0	42	1	0.2
1	16	2	1.6
2	6	3	2.7
3	5	5	5.6
4	6	8	11.5

```
pipeline = [unwind, match, group, bucket, project]
results = list(launches_stage.aggregate(pipeline))

labels = [str(r["flight_range"]) for r in results]
core_counts = [r["core_count"] for r in results]
avg_landings = [r["avg_landing_success"] for r in results]

fig, ax1 = plt.subplots(figsize=(8, 4))

bars = ax1.bar(labels, core_counts, color="#9b59b6", alpha=0.8, label="Number of Cores")
ax1.set_xlabel("Number of Flights (reuse bucket)")
ax1.set_ylabel("Number of Cores", color="#9b59b6")
ax1.tick_params(axis="y", labelcolor="#9b59b6")

ax2 = ax1.twinx()
ax2.plot(labels, avg_landings, color="#f39c12", marker="o", linewidth=2, label="Avg Landing Successes
ax2.set_ylabel("Avg Landing Successes", color="#f39c12")
ax2.tick_params(axis="y", labelcolor="#f39c12")

fig.legend(loc="upper right", bbox_to_anchor=(0.95, 0.95))
plt.tight_layout()
```

```
plt.savefig("Personal_project_files/images/core_reuse.png", dpi=150)
plt.show()
```

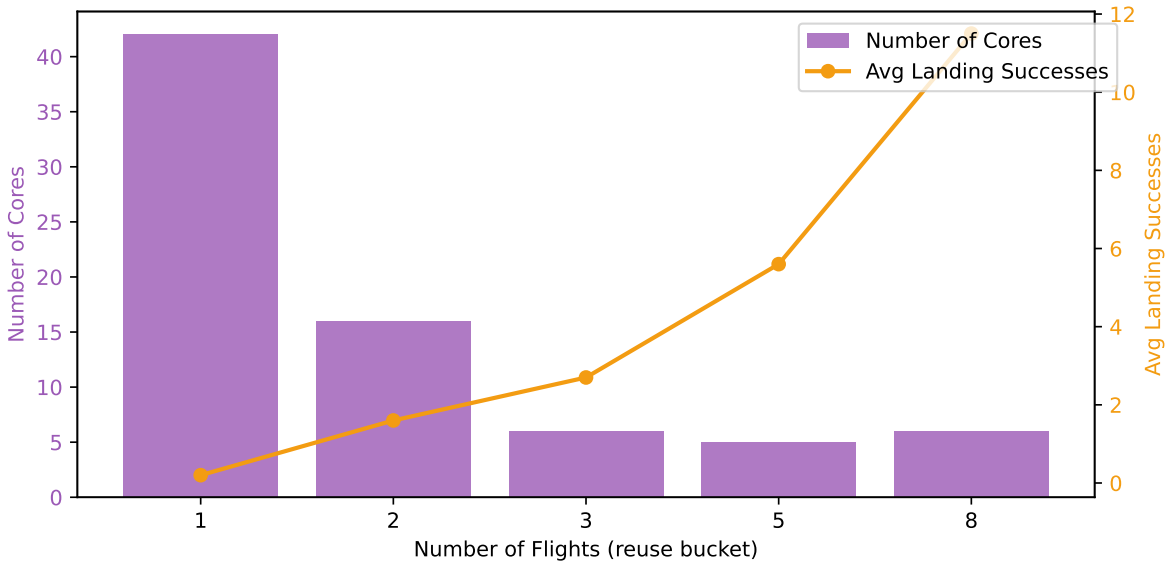

Figure 6: Core Reuse Distribution and Landing Success

The bar-and-line chart reveals that the majority of cores flew only once. These include early expendable missions and newer cores that have not yet been reused. As the number of flights increases, the count of cores drops sharply, but the average landing successes per core rises steeply, reaching 11.5 for the most-reused bucket. This confirms that reused boosters are not only surviving multiple missions but consistently landing successfully, validating SpaceX's reusability model. The 6 cores in the 8+ bucket represent the fleet's most battle-tested hardware and are central to SpaceX's cost-reduction strategy.

6.6 Fail rate

This pipeline focuses specifically on landing failures. After unwinding cores and filtering to only those where a landing was actually attempted (`landing_attempt: True`) and a definitive outcome exists, it performs a `$lookup` to join rocket details and groups the results by rocket name. The `$group stage` counts total landing attempts, failures, and successes per rocket type. A `$addFields` stage then computes the landing fail rate as a percentage. By joining with rockets_stage via `$lookup` and `$unwind`, the pipeline attributes each landing failure to a specific rocket type, enabling a direct comparison of landing reliability between Falcon 9 and Falcon Heavy.

```
# only look at core entries where a landing was actually attempted
unwind = {"$unwind": "$cores"}

# exclude cores where no landing was attempted or rocket info is missing
match = {"$match": {
```

```
    "cores.landing_attempt": True,
    "cores.landing_success": {"$in": [True, False]},
    "rocket": {"$ne": None}
}}

# join rocket details to get the rocket name
lookup = {"$lookup": {
    "from": "rockets_stage",
    "localField": "rocket",
    "foreignField": "_id",
    "as": "rocket_info"
}}

# flatten rocket_info array
unwind2 = {"$unwind": "$rocket_info"}

# group by rocket name and count landing attempts and failures
group = {"$group": {
    "_id": "$rocket_info.name",
    "landing_attempts": {"$sum": 1},
    "landing_failures": {"$sum": {
        "$cond": [{"$eq": ["$cores.landing_success", False]}, 1, 0]
    }},
    "landing_successes": {"$sum": {
        "$cond": [{"$eq": ["$cores.landing_success", True]}, 1, 0]
    }}
}}

# compute landing fail rate
add_fields = {"$addFields": {
    "landing_fail_rate": {
        "$round": [
            {"$multiply": [
                {"$divide": ["$landing_failures", "$landing_attempts"]},
                100
            ]},
            1
        ]
    }
}}

# clean up output fields
project = {"$project": {
    "_id": 0,
    "rocket": "$_id",
    "landing_attempts": 1,
    "landing_failures": 1,
    "landing_successes": 1,
```

```
    "landing_fail_rate": 1
}}

# sort by most attempts first
sort = {"$sort": {"landing_attempts": -1}}
```

```
pipeline = [unwind, match, lookup, unwind2, group, add_fields, project, sort]
cursor = launches_stage.aggregate(pipeline)
pd.DataFrame(cursor)
```

Table 6: Landing Fail Rate per Rocket Type

	landing_attempts	landing_failures	landing_successes	landing_fail_rate	rocket
0	152	11	141	7.2	Falcon 9
1	9	2	7	22.2	Falcon Heavy

```
pipeline = [unwind, match, lookup, unwind2, group, add_fields, project, sort]
results = list(launches_stage.aggregate(pipeline))

rockets = [r["rocket"] for r in results]
fail_rates = [r["landing_fail_rate"] for r in results]
attempts = [r["landing_attempts"] for r in results]

fig, ax = plt.subplots(figsize=(7, 3))
bars = ax.bar(rockets, fail_rates,
              color=["#e74c3c" if r > 20 else "#e67e22" if r > 10 else "#2ecc71" for r in fail_rates]

for bar, attempt, rate in zip(bars, attempts, fail_rates):
    ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.5,
            f"{rate}%\n({attempt} attempts)", ha="center", va="bottom", fontsize=9)

ax.set_ylabel("Landing Fail Rate (%)")
ax.set_xlabel("Rocket Type")
ax.set_ylim(0, max(fail_rates) + 15)
plt.tight_layout()
plt.savefig("Personal_project_files/images/landing_fail_rate.png", dpi=150)
plt.show()
```

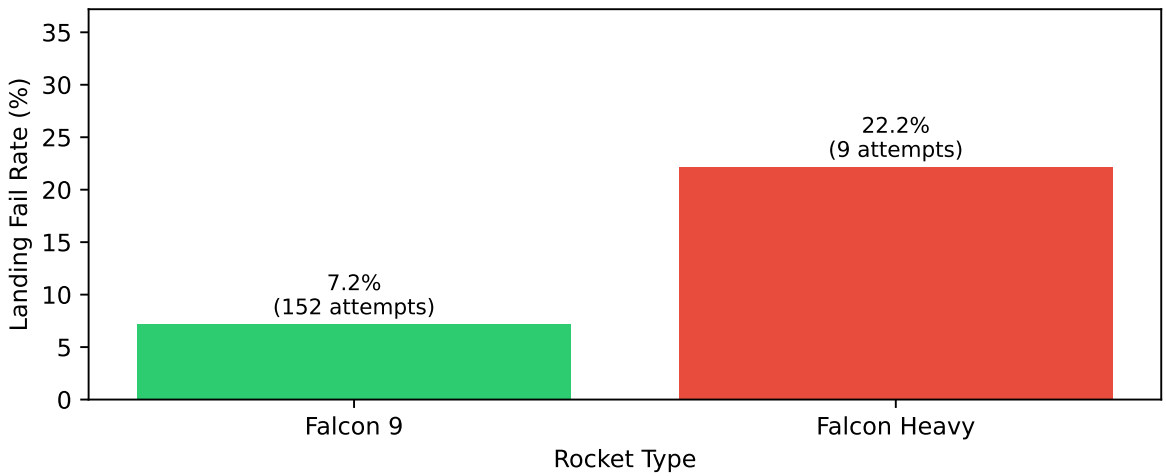

Figure 7: Landing Fail Rate per Rocket Type

The bar chart shows that Falcon 9 has a landing fail rate of only 7.2% across 152 attempts. Meaning 141 out of 152 boosters were successfully recovered. Falcon Heavy, with just 9 landing attempts, has a notably higher fail rate of 22.2%. The higher Falcon Heavy rate is partly explained by the complexity of recovering three boosters per mission and the smaller sample size. The colour coding (green for Falcon 9, red for Falcon Heavy) draws attention to this disparity. Nevertheless, both rates are impressive given that routine orbital-class booster recovery was considered impractical before SpaceX pioneered it.